# Conceptual Database Schema

## Scope: Account creation, verification, admin review

**Framework:** Flask

**Storage:** SQLite (conceptual; MySQL-compatible)

**Agencies:** Not included yet

---

## 1. Core Principles (Locked)

- Every user has **one role**
- Verification is **request-based**
- Cooldown is **derived**, not stored
- Rejected requests are **archived**
- No duplicate active requests allowed
- Documents are **referenced**, not embedded

---

## 2. Tables Overview

| Table | Purpose |
|---|---|
| `users` | Authentication & identity |
| `technician_profiles` | Technician-specific data |
| `business_profiles` | Business-specific data |
| `verification_requests` | Verification lifecycle |
| `verification_flags` | Suspicious indicators |
| `uploaded_documents` | Document metadata |
| `notifications` | User-facing messages |
| `admin_actions` | Audit trail (optional but recommended) |

---

## 3. Table Definitions (Conceptual)

## 3.1 `users`

**Purpose:**

Core authentication identity.

**Fields**

- `id` (PK)
- `email` (unique)
- `password_hash`
- `role`
  - ENUM: `ADMIN`, `TECHNICIAN`, `BUSINESS`
- `is_active` (boolean)
- `created_at`
- `last_login_at`

**Rules**

- One row per login identity
- Admin users **do not** require verification requests

---

## 3.2 `technician_profiles`

**Purpose:**

Stores technician-specific details.

**Fields**

- `id` (PK)
- `user_id` (FK → users.id, unique)
- `full_name`
- `skills` (structured text / JSON later)
- `bio` (optional)
- `created_at`

**Rules**

- Exists only if `users.role = TECHNICIAN`

- One-to-one with `users`

---

## 3.3 `business_profiles`

**Purpose:**

Stores business-specific details.

**Fields**

- `id` (PK)
- `user_id` (FK → users.id, unique)
- `company_name`
- `registration_identifier`
- `created_at`

**Rules**

- Exists only if `users.role = BUSINESS`
- One-to-one with `users`

---

## 3.4 `verification_requests`

**Purpose:**

Tracks verification lifecycle.

**Fields**

- `id` (PK)
- `user_id` (FK → users.id)
- `user_role` (`TECHNICIAN` / `BUSINESS`)
- `status`
  - ENUM: `PENDING`, `APPROVED`, `REJECTED`
- `submitted_at`
- `reviewed_at` (nullable)
- `reviewed_by_admin_id` (FK → users.id, nullable)
- `rejection_reason` (nullable)

- `rejected_at` (nullable)
- `cooldown_until` (nullable, computed & stored OR computed dynamically)

**Rules**

- Only **one active request** per user:
  - Active = `PENDING` OR (`REJECTED` AND now < cooldown_until)
- `APPROVED` is terminal
- Old requests remain for audit

---

## 3.5 `verification_flags`

**Purpose:**

Stores flags attached to a verification request.

**Fields**

- `id` (PK)
- `verification_request_id` (FK)
- `flag_type`
  - Examples:
    - `SUSPICIOUS_NAME_FORMAT`
    - `REPEATED_SUBMISSION_IP`
    - `UNUSUAL_DOCUMENT_PATTERN`
- `severity`
  - ENUM: `LOW`, `MEDIUM`, `HIGH`
- `description`
- `created_at`

**Rules**

- Zero or more flags per request
- Flags **do not** block approval

---

## 3.6 `uploaded_documents`

**Purpose:**

Metadata for uploaded `.pdf` / `.docx` files.

**Fields**

- `id` (PK)
- `verification_request_id` (FK)
- `uploaded_by_user_id` (FK → users.id)
- `document_type`
  - Examples:
    - `CERTIFICATION`
    - `BUSINESS_REGISTRATION`
- `original_filename`
- `stored_filename`
- `file_extension` (`pdf`, `docx`)
- `file_size`
- `uploaded_at`

**Rules**

- Only `.pdf` and `.docx` allowed
- Stored filename must be sanitized
- Files linked to verification requests, not users directly

---

## 3.7 `notifications`

**Purpose:**

User-facing system messages.

**Fields**

- `id` (PK)
- `user_id` (FK)
- `type`
  - Examples:

        - `VERIFICATION_APPROVED`
        - `VERIFICATION_REJECTED`
- `message`
- `is_read` (boolean)
- `created_at`
- `read_at` (nullable)

**Rules**

- Notifications persist across sessions
- Shown on login/dashboard

---

## 3.8 `admin_actions` (Optional but Strongly Recommended)

**Purpose:**

Audit trail for admin decisions.

**Fields**

- `id` (PK)
- `admin_user_id` (FK → users.id)
- `action_type`
    - Examples:
        - `APPROVE_VERIFICATION`
        - `REJECT_VERIFICATION`
- `target_verification_request_id` (FK)
- `timestamp`
- `notes` (optional)

**Rules**

- Read-only log
- Never modified or deleted

---

## 4. Relationships Summary (Textual ER)

```
users
 ├──1 →1 technician_profiles (if TECHNICIAN)
 ├──1 →1 business_profiles (if BUSINESS)
 ├──1 → many verification_requests
 ├──1 → many notifications

verification_requests
 ├──1 → many verification_flags
 ├──1 → many uploaded_documents
 ├── reviewed_by → users (ADMIN)

admin_actions
 └── many → users (ADMIN)
```

---

## 5. Cooldown Logic (Schema-Aligned)

Cooldown is **not a state**, but computed as:

```
cooldown_active =
status == REJECTED
   AND now < cooldown_until
```

- `cooldown_until` may be:
  - Stored explicitly **OR**
  - Computed from `rejected_at + COOLDOWN_DURATION`

Both are valid as long as **no COOLDOWN enum exists**.

---

## 6. Constraints That Must Be Enforced in Code

(Not DB-level only)

- One active verification request per user
- Only admins can update verification status

- Approved requests cannot be modified
- Rejected requests cannot be approved
- Uploads restricted to `.pdf` / `.docx`

---

## 7. Why This Schema Is Correct

This schema:

- Matches your routes exactly
- Matches your state machine exactly
- Prevents invalid transitions
- Supports admin review cleanly
- Prevents spam without auto-reject
- Scales to agencies later without breaking tables